TD 2 : Structure de données

Exercice 1 : Saisie de notes

Écrire un programme qui demande à l'utilisateur de saisir **cinq notes**.
Ces notes seront stockées dans une liste appelée notes.
À la fin, le programme devra afficher la liste complète des notes saisies.

Exercice 2 : Calculs sur les notes

À partir de la liste notes obtenue précédemment, écrire un programme qui calcule et affiche :

- la somme des notes,
- la moyenne,
- la note maximale,
- la note minimale.

Exercice 3 : Nombres pairs et impairs

Écrire un programme qui demande à l'utilisateur de saisir **six nombres** et de les stocker dans une liste appelée nombres.
Le programme devra ensuite afficher :

- les nombres pairs,
- les nombres impairs.

Exercice 4 : Modification des valeurs

Écrire un programme qui demande à l'utilisateur de saisir **cinq nombres**.
Le programme devra ensuite :

- multiplier chaque valeur par 2,
- remplacer les nombres pairs par 0,
- afficher la liste finale obtenue.

Exercice 5 : Recherche dans un tableau

Écrire un programme qui demande à l'utilisateur de saisir **quatre produits**, puis de les stocker dans une liste appelée produits.
Ensuite, le programme devra demander à l'utilisateur de saisir un produit à

rechercher.
Il devra vérifier si ce produit existe dans la liste et afficher sa position s'il est trouvé.

Exercice 6 : Tri d'une liste

Écrire un programme qui demande à l'utilisateur de saisir **cinq nombres**.
Le programme devra ensuite :

- trier la liste par ordre croissant,
- puis trier la liste par ordre décroissant,
- afficher les résultats obtenus.

Exercice 7 : Fusion de listes

Écrire un programme qui demande à l'utilisateur de saisir **trois nombres** pour une première liste liste1, puis **trois nombres** pour une seconde liste liste2.
Le programme devra ensuite créer une nouvelle liste contenant les éléments des deux listes et afficher cette liste fusionnée.

Exercice 8 : Comptage d'occurrences

Écrire un programme qui demande à l'utilisateur de saisir **huit nombres** et de les stocker dans une liste.
Le programme devra ensuite demander à l'utilisateur de saisir un nombre à rechercher et afficher le nombre de fois où cette valeur apparaît dans la liste.

Exercice 9 : Filtrage des données

Écrire un programme qui demande à l'utilisateur de saisir **six âges**.
Le programme devra ensuite créer :

- une liste contenant les personnes majeures (âge supérieur ou égal à 18),
- une liste contenant les personnes mineures.
  Les deux listes devront être affichées.

Exercice 10 : Gestion de dépenses

Écrire un programme qui demande à l'utilisateur de saisir **six dépenses (en FCFA)**.
Ces valeurs seront stockées dans une liste appelée depenses.

Le programme devra ensuite :

- calculer le total des dépenses,
- calculer la moyenne,
- déterminer la dépense maximale et la dépense minimale,
- afficher toutes les dépenses supérieures à 2000 FCFA.

Exercice 11 : Gestion d'une liste de dépenses (Listes) – ⭐

On considère la liste suivante représentant les dépenses journalières (en FCFA) d'un étudiant :

depenses = [1500, 2000, 500, 3000, 2500, 1000]

1. Afficher la première et la dernière dépense
2. Calculer le total des dépenses
3. Calculer la dépense moyenne
4. Ajouter une nouvelle dépense de 1800 FCFA
5. Supprimer la dépense de 500 FCFA
6. Trier la liste par ordre croissant

Exercice 12 : Informations sur des étudiants (Dictionnaires)

On dispose du dictionnaire suivant :

```
etudiant = {
    "nom": "Issa",
    "age": 22,
    "notes": [12, 15, 14]
}
```

1. Afficher le nom de l'étudiant
2. Calculer la moyenne des notes
3. Ajouter une nouvelle note : 16
4. Modifier l'âge de l'étudiant à 23 ans
5. Ajouter une nouvelle clé "ville" avec la valeur "Ouagadougou"

6. Afficher toutes les clés du dictionnaire

Exercice 13 : Gestion d'un carnet de contacts

On veut représenter un carnet de contacts :

```
contacts = {
    "Ali": "70123456",
    "Mariam": "65000000",
    "Paul": "78000000"
}
```

1. Afficher le numéro de Mariam
2. Ajouter un nouveau contact "Fatou": "76000000"
3. Supprimer le contact "Paul"
4. Vérifier si "Ali" existe dans le carnet
5. Afficher tous les noms des contacts
6. Afficher tous les numéros de téléphone

Exercice 14 : Analyse de températures

On a relevé les températures d'une semaine :

```
temperatures = [30, 32, 35, 33, 31, 29, 28]
jours = ("Lundi", "Mardi", "Mercredi", "Jeudi", "Vendredi", "Samedi",
"Dimanche")
```

1. Afficher la température du mercredi
2. Trouver la température maximale et minimale
3. Calculer la moyenne des températures
4. Afficher les jours où la température dépasse 32°C
5. Associer chaque jour à sa température (affichage sous forme : "Lundi : 30°C")

Exercice 15

On dispose d'une liste de transactions Mobile Money. Chaque transaction est représentée par un dictionnaire contenant :

- date : date de la transaction
- sms : message reçu

- montant : montant de la transaction (en FCFA)
- solde : solde après la transaction
- type : "recu" ou "paiement"

```
transactions = [
    {
        "date": "2024-03-01",
        "sms": "Vous avez recu 5000 FCFA",
        "montant": 5000,
        "solde": 15000,
        "type": "recu"
    },
    {
        "date": "2024-03-02",
        "sms": "Paiement de 2000 FCFA effectue",
        "montant": 2000,
        "solde": 13000,
        "type": "paiement"
    },
    {
        "date": "2024-03-03",
        "sms": "Vous avez recu 15000 FCFA",
        "montant": 15000,
        "solde": 28000,
        "type": "recu"
    },
    {
        "date": "2024-03-04",
        "sms": "Paiement de 3000 FCFA effectue",
        "montant": 3000,
        "solde": 25000,
        "type": "paiement"
    }
]
```

1. Exploration des données

a. Afficher le nombre total de transactions

b. Afficher la première transaction

c. Afficher toutes les dates des transactions

2. Filtrage des données

a. Créer une liste contenant uniquement les transactions de type "recu"

b. Créer une liste contenant uniquement les transactions de type "paiement"

c. Afficher les montants des transactions reçues

3. **Calculs**

a. Calculer le **total reçu**
b. Calculer le **total dépensé**
c. Calculer le **solde final** (en utilisant les montants)
d. Vérifier si ce solde correspond au dernier solde de la liste

4. **Analyse avancée**

a. Trouver la transaction avec le montant le plus élevé
b. Afficher toutes les transactions supérieures à 5000 FCFA
c. Compter le nombre de transactions de chaque type

5. **Création d'un résumé**

Créer un dictionnaire resume contenant :

```
resume = {
    "total_recu": ...,
    "total_depense": ...,
    "solde_final": ...,
    "nombre_transactions": ...,
    "nombre_recu": ...,
    "nombre_paiement": ...
}
```

Puis afficher ce dictionnaire.